Paso 3 · Receta de integración local

MediNet-Core · Gestión de Pacientes y Admisión (PAC) · Grupo 6
EVA02 Desarrollo Seguro (CI3064) · INACAP Temuco
Edición explicada para principiantes · 29 de octubre de 2026

Qué conseguirás al terminar

Vas a tomar el proyecto del paso 2, incorporar el código de pacientes, actualizar la estructura de PostgreSQL sin borrar la base y poner la API a funcionar en tu equipo. Después crearás, consultarás y modificarás un paciente inventado para comprobarlo.

Esta es una receta para seguir en orden. No necesitas memorizar los comandos: lee qué hacen, ejecútalos y compara el resultado antes de avanzar. No ejecutes todos los bloques de una sola vez.

Qué necesitas tener a mano

- Tu carpeta MediNet-Core del paso 2 y acceso a su contraseña administrativa.
- Este PDF y el archivo Paso3_archivos.zip de la entrega anterior.
- Un editor de texto, un navegador y una terminal.
- Python 3.14 y el motor de contenedores que usaste en el paso 2.
- Conexión a Internet para instalar las dependencias si todavía no están disponibles.

> Trabajaremos con personas inventadas. El paquete no contiene .env, contraseñas ni pacientes. Cada integrante conserva su propia configuración local. Esta guía explica cómo reproducir el trabajo; no afirma haber ejecutado estas comprobaciones en el equipo de cada compañero.

Antes de empezar · El mapa de la receta

Las cuatro piezas que vas a conectar

Navegador → API → PostgreSQL → almacenamiento. En el navegador usarás Swagger para enviar una solicitud. La API revisará los datos y, cuando corresponda, pedirá a PostgreSQL que los guarde o consulte. PostgreSQL mantiene sus archivos en un volumen para conservarlos aunque detengas el contenedor.

Palabras que usaremos

- **Terminal:** ventana donde escribes órdenes para el sistema.
- **Comando:** una orden que ejecutas presionando Enter.
- **API:** programa que recibe solicitudes y devuelve respuestas. En este proyecto trabaja con pacientes.
- **Base de datos:** lugar organizado donde guardamos información; el motor que la administra se llama PostgreSQL.
- **Contenedor:** entorno donde se ejecuta PostgreSQL con su configuración.
- **Volumen:** almacenamiento que conserva los datos del contenedor.
- **Puerto:** número que identifica un servicio. La base usará 5433 y nuestra API 8001.
- **localhost o 127.0.0.1:** tu propio equipo.
- **Entorno virtual:** carpeta .venv que guarda las librerías de Python de este proyecto.
- **Migración:** cambio controlado de la estructura de la base; Alembic lleva su registro.
- **Rol:** cuenta de PostgreSQL con permisos concretos.

> Los ejemplos principales usan Linux y Podman, que es lo utilizado en este workspace. Si tu paso 2 usa Docker, lee primero la página siguiente. No instales otro motor pensando que encontrará automáticamente tus datos.

Cómo leer y ejecutar los comandos

Dónde escribir cada cosa

Cuando diga **Terminal A** o **Terminal B**, escribe el comando en una ventana de terminal del sistema. Cuando diga **dentro de psql**, estarás en el programa de PostgreSQL. Cuando diga **Swagger**, usarás el navegador. Un bloque JSON se pega en Swagger, no en la terminal.

Reglas para copiar

1. Copia un comando a la vez. No copies símbolos como $ o el texto que aparece antes del cursor de tu terminal.
2. Si una línea termina en barra invertida (\), el comando continúa abajo. Copia el bloque completo sin agregar texto después de esa barra.
3. Las rutas con espacios deben ir entre comillas.
4. Los ejemplos como /ruta/de/tu/MediNet-Core se reemplazan por tu ruta real.
5. Si una contraseña no muestra letras ni asteriscos al escribirla, es normal: escribe y presiona Enter.
6. No uses sudo para editar el código o instalar librerías dentro de .venv.

Si tu paso 2 usa Docker

En comandos de la base, cambia **podman** por **docker**, y **podman-compose** por **docker compose**. Ejemplo: podman-compose up -d pasa a ser docker compose up -d. Conserva el motor y el volumen del paso 2. El runner automatizado del proyecto sí requiere Podman; se explica al final.

> Esta guía presupone los nombres estándar: contenedor medinet_pac_db, base medinet_pac y administrador medinet_pac. Si tu instalación tiene otros nombres, no los cambies a ciegas: compáralos con tu docker-compose.yml antes de continuar. No se certificó la instalación en Windows; para seguir estos comandos allí se necesita un entorno Linux como WSL y un motor operativo.

1. Entrar a la carpeta correcta

Qué vamos a hacer y para qué

Ubicarnos en la carpeta que contiene el proyecto. Así evitamos instalar o ejecutar cosas en otro lugar.

Antes de hacerlo, entiende esto

cd significa cambiar de carpeta; pwd muestra dónde estás; ls lista su contenido. La raíz del proyecto es la carpeta que contiene app, README.md y docker-compose.yml.

Hazlo en este orden

En la Terminal A: abre tu gestor de archivos, busca MediNet-Core y copia su ruta. Sustituye la ruta del ejemplo:

```
cd "/ruta/de/tu/MediNet-Core"
pwd
ls
python3 --version
```

Busca también el archivo .env en tu editor o activa “mostrar archivos ocultos” en el gestor de archivos. Su nombre empieza con un punto; por eso puede no verse en un listado normal.

Cómo saber si funcionó

pwd termina en MediNet-Core; ls muestra app y docker-compose.yml. Python debe ser 3.14.x para seguir el entorno del proyecto.

Si no sale como esperabas

Si cd dice que la carpeta no existe, corrige la ruta. Si Python no está instalado o tiene otra versión, resuelve primero el requisito del paso 1; no reemplaces el Python del sistema a ciegas.

2. Conservar una copia antes de cambiar

Qué vamos a hacer y para qué

Proteger el trabajo del paso 2 antes de reemplazar archivos o modificar el esquema.

Antes de hacerlo, entiende esto

Respaldar archivos y respaldar la base son cosas distintas. La carpeta contiene código; el volumen contiene los pacientes. Copiar solo la carpeta no equivale a guardar los datos de PostgreSQL.

Hazlo en este orden

1. Desde el gestor de archivos, copia tu carpeta MediNet-Core a otra carpeta local, fuera del proyecto. Nómbrala MediNet-Core-antes-paso3. Esa copia puede contener .env: no la compartas.

2. En la Terminal A, comprueba que tu contenedor anterior está activo:

```
podman ps -a
```

3. Si medinet_pac_db aparece detenido, enciéndelo sin recrearlo:

```
podman start medinet_pac_db
```

4. Si el contenedor no existe y nunca creaste la base, no hay datos que respaldar: continúa al paso 3 y sigue después la rama de base nueva. Si esperabas tener datos, detente y localiza primero tu motor y volumen.

Cómo saber si funcionó

Tienes una copia de los archivos y has identificado el contenedor correcto. Si existe, aparece activo en podman ps.

Si no sale como esperabas

No uses comandos de limpieza global ni down -v. Si tu base está en Docker, consulta docker ps -a; no busques sus contenedores en Podman.

2.1. Respaldar la base existente

Qué vamos a hacer y para qué

Crear un archivo con el contenido y la estructura de PostgreSQL antes de aplicar migraciones.

Antes de hacerlo, entiende esto

pg_dump prepara una copia lógica. -Fc usa un formato de respaldo de PostgreSQL. El símbolo > guarda la salida en un archivo de tu equipo. chmod limita quién puede leerlo.

Hazlo en este orden

Solo si ya tienes la base del paso 2. En la Terminal A, desde la raíz del proyecto, ejecuta cada comando:

```
mkdir -p respaldos_locales
chmod 700 respaldos_locales
podman exec medinet_pac_db pg_dump -U medinet_pac \
  -d medinet_pac -Fc > respaldos_locales/pac_antes_paso3.dump
chmod 600 respaldos_locales/pac_antes_paso3.dump
ls -lh respaldos_locales/pac_antes_paso3.dump
```

mkdir crea la carpeta. podman exec ejecuta pg_dump dentro del contenedor; el archivo se guarda en respaldos_locales de tu equipo. No lo compartas con tus compañeros: podría contener datos.

Abre .gitignore con tu editor y añade una línea respaldos_locales/ si todavía no existe. Así Git no propondrá subir esa carpeta. Si reemplazas .gitignore en el siguiente paso, vuelve a comprobar esa línea.

Cómo saber si funcionó

pg_dump termina sin error y ls muestra un archivo no vacío. Conserva esa copia. No estamos restaurándola: solo preparándola por si hiciera falta.

Si no sale como esperabas

Si pg_dump informa un error, no consideres válido el respaldo aunque exista un archivo. Comprueba contenedor, base y usuario; no avances con cambios sobre una base que querías proteger sin completar el respaldo.

3. Incorporar los archivos de la entrega

Qué vamos a hacer y para qué

Agregar la implementación que hicimos después del paso 2.

Antes de hacerlo, entiende esto

Esta receta configura código ya construido. El ZIP incluye ese código porque el proyecto de origen todavía no tiene commits ni push; no hay una revisión remota común que podamos indicar con git pull.

Hazlo en este orden

1. Descarga Paso3_archivos.zip y descomprímelo en una carpeta temporal, por ejemplo Descargas/Paso3_temporal.

2. Abre la carpeta MediNet-Core-Paso3 que aparece dentro. Verás app, migrations, scripts, tests y archivos de configuración.

3. Con tu copia de seguridad lista, copia el contenido de esa carpeta sobre tu MediNet-Core original. Si tienes cambios propios, compáralos con tu editor antes de reemplazarlos.

4. Conserva tu .env, .git y .venv locales. El ZIP no los incluye. Copia también .env.example y .gitignore, que pueden estar ocultos. Reincorpora respaldos_locales/ a .gitignore si corresponde.

5. En la Terminal A comprueba:

```
ls app/run_local.py app/schemas.py alembic.ini
ls migrations/versions
ls scripts tests
```

Cómo saber si funcionó

Están run_local.py, schemas.py y las migraciones 0001_existing_patients.py y 0002_prefix_indexes.py.

Si no sale como esperabas

Si no aparecen, probablemente copiaste la carpeta envolvente en lugar de su contenido. Debe existir MediNet-Core/app, no MediNet-Core/MediNet-Core-Paso3/app.

4. Preparar el entorno de Python

Qué vamos a hacer y para qué

Instalar las librerías exactas que usa esta versión del proyecto.

Antes de hacerlo, entiende esto

Python ejecuta el programa. pip instala librerías. El entorno virtual .venv evita mezclar estas librerías con las de otros proyectos. requirements-dev.txt incluye ejecución y pruebas.

Hazlo en este orden

En la Terminal A, dentro de MediNet-Core:

```
python3 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install -r requirements-dev.txt
python -m pip check
podman-compose --version
```

El primer comando crea el entorno si no existe. Si ya tenías un entorno compatible con Python 3.14, puedes activarlo y continuar con la instalación. source lo activa únicamente en esta terminal. Cada terminal nueva necesita activación antes de usar python sin ruta completa.

La instalación puede tardar y requiere Internet. No cambies las versiones de los archivos requirements para resolver problemas sin revisarlo con el equipo.

Cómo saber si funcionó

Python muestra 3.14.x; la instalación termina sin error y pip check indica que no hay requisitos incompatibles. Es frecuente que el prompt muestre (.venv).

Si no sale como esperabas

Si aparece externally-managed-environment, revisa que activaste .venv. Si faltan paquetes o falló una descarga, corrige ese error y repite la instalación; no uses sudo pip.

5. Revisar .env sin perder la contraseña

Qué vamos a hacer y para qué

Preparar los datos que usan Compose y las migraciones para entrar como administrador.

Antes de hacerlo, entiende esto

Un archivo .env es un archivo de texto con pares NOMBRE=valor. No es código Python. POSTGRES_PASSWORD corresponde al administrador; la API tendrá otra cuenta y otra contraseña.

Hazlo en este orden

1. Si ya tenías .env del paso 2, **consérvalo**. Ábrelo en tu editor y comprueba los nombres de las variables. No pegues su contenido en un chat.

2. Solo si NO existe, copia .env.example y genera una contraseña:

```
cp -n .env.example .env
python -c 'import secrets; print(secrets.token_urlsafe(32))'
```

3. En tu editor, completa el .env de una base nueva con esta estructura. Reemplaza el marcador de contraseña por el valor generado:

```
POSTGRES_DB=medinet_pac
POSTGRES_USER=medinet_pac
POSTGRES_PASSWORD=TU_CONTRASENA_ADMINISTRATIVA_LOCAL
DB_HOST=127.0.0.1
DB_PORT=5433
```

4. Guarda el archivo y limita su lectura:

```
chmod 600 .env
```

Cómo saber si funcionó

El .env contiene los cinco valores necesarios y .gitignore incluye .env. No contiene literalmente TU_CONTRASENA_ADMINISTRATIVA_LOCAL.

Si no sale como esperabas

Editar .env no cambia la contraseña de una base ya inicializada. Si olvidaste la contraseña del paso 2, no inventes una nueva en este archivo: recupera o revisa la credencial antes de migrar.

6. Encender y reconocer PostgreSQL

Qué vamos a hacer y para qué

Dejar disponible la base local que utilizará la API.

Antes de hacerlo, entiende esto

Compose lee docker-compose.yml y .env. up -d arranca el servicio en segundo plano. El puerto 5433 de tu equipo se dirige al 5432 del contenedor. El volumen guarda los datos.

Hazlo en este orden

En la Terminal A:

```
podman-compose up -d
podman-compose ps
podman exec medinet_pac_db pg_isready \
  -U medinet_pac -d medinet_pac
```

Si estás continuando el paso 2, Compose debe conservar su volumen. Si es una base realmente nueva, inicializará el volumen con los valores de .env. No cambies el nombre del proyecto o del volumen para intentar arreglar un inicio fallido.

pg_isready pregunta si PostgreSQL está disponible. Todavía no comprueba la contraseña de la cuenta que usará la API.

Cómo saber si funcionó

El servicio aparece activo y pg_isready responde accepting connections. La base debe estar publicada en 127.0.0.1:5433.

Si no sale como esperabas

Si el puerto está ocupado, identifica qué servicio lo usa antes de detenerlo. Si Docker administraba la base, usa docker compose. No ejecutes down -v: eliminaría los datos del volumen.

7. Actualizar la estructura con Alembic

Qué vamos a hacer y para qué

Aplicar los cambios del paso 3 conservando los pacientes del paso 2.

Antes de hacerlo, entiende esto

Una migración cambia la estructura de la base de forma registrada. Alembic sabe qué cambios ya se aplicaron. head significa la última revisión disponible; en esta entrega es 0002.

Hazlo en este orden

En la Terminal A, con .venv activo:

```
python -m alembic upgrade head
python -m alembic current
python -m alembic check
```

upgrade head aplica lo pendiente. current informa en qué revisión quedó la base. check compara el modelo de Python con el esquema migrado.

Qué hace 0001: si no existe patients, la crea. Si ya existe del paso 2, revisa sus columnas, restricciones e índice antes de adoptarla.

Qué hace 0002: agrega índices para buscar por apellido paterno y materno. Un índice ayuda a localizar filas sin revisar toda la tabla en ciertos tipos de búsqueda.

El comando antiguo python -m app.crear_tablas sigue disponible, pero ahora ejecuta Alembic. No necesitas ejecutarlo además del upgrade.

Cómo saber si funcionó

current muestra 0002 como head y check termina con No new upgrade operations detected.

Si no sale como esperabas

Si falla por contraseña, revisa .env y posibles variables exportadas que lo sobrescriban. Si falla por esquema incompatible, no borres la tabla ni uses stamp para forzarlo: conserva el error y compara tu esquema con el equipo. 0001 bloquea su reversión destructiva.

8. Entender y comprobar el usuario de API

Qué vamos a hacer y para qué

Separar la cuenta que administra la base de la cuenta que usa el programa.

Antes de hacerlo, entiende esto

El administrador medinet_pac puede cambiar el esquema. medinet_pac_api debe poder consultar, crear y actualizar pacientes, sin poder borrar la tabla. Esto limita lo que puede hacer una conexión de la API.

Hazlo en este orden

En la Terminal A: pregunta si el rol ya existe:

```
podman exec medinet_pac_db psql -U medinet_pac \
  -d medinet_pac -tAc \
  "SELECT rolname FROM pg_roles WHERE rolname='medinet_pac_api';"
```

psql es el cliente de PostgreSQL. -U selecciona el usuario y -d la base. SELECT consulta información; aquí consultamos los roles, no los pacientes.

Si aparece medinet_pac_api: ya existe. Omite la creación de la página siguiente y ve a la contraseña.

Si no aparece ninguna fila y no hubo error: todavía no existe. Créalo una sola vez con el siguiente paso.

> No confundas una salida vacía válida con un error de conexión. Si el comando muestra un error, no sabemos aún si el rol existe. Resuelve el error antes de decidir.

Cómo saber si funcionó

Sabes si necesitas crear el rol o si debes reutilizar el existente.

Si no sale como esperabas

Si el administrador o la base tienen otro nombre, el script estándar no se puede aplicar sin revisarlo: verifica la configuración con el equipo. No uses la cuenta limitada para crear roles.

8.1. Crear el rol y su contraseña

Qué vamos a hacer y para qué

Dar al programa sus permisos y establecer una contraseña exclusiva para esa cuenta.

Antes de hacerlo, entiende esto

El script SQL agrupa la creación y los permisos en una transacción: se confirman juntos. ON_ERROR_STOP hace que psql se detenga ante un error. El rol se crea sin contraseña y luego la definimos con entrada oculta.

Hazlo en este orden

Solo si el rol no existe, en la Terminal A:

```
podman exec -i medinet_pac_db psql -U medinet_pac \
  -d medinet_pac -v ON_ERROR_STOP=1 \
  < scripts/pac_runtime_role.sql
```

El símbolo < envía el archivo SQL al programa. Espera el COMMIT. No repitas este script si el rol ya existe: no está diseñado para ejecutarse dos veces.

Ahora define la contraseña del rol nuevo. En un rol existente, hazlo solo si necesitas configurarla o corregirla:

```
podman exec -it medinet_pac_db psql \
  -U medinet_pac -d medinet_pac
```

El prompt cambia: ahora estás dentro de psql. Escribe estas órdenes por separado:

```
\password medinet_pac_api
\q
```

La primera pide una contraseña y su confirmación; la segunda sale de psql. Usa una distinta a la administrativa y guárdala en tu gestor local de contraseñas.

Cómo saber si funcionó

El script termina en COMMIT y psql confirma el cambio de contraseña. Al salir vuelves al prompt del sistema.

Si no sale como esperabas

El script revoca TEMPORARY de PUBLIC en medinet_pac. Está pensado para la base local dedicada a PAC; si compartes esa base con otras aplicaciones, revisa ese impacto. Si el rol ya estaba aplicado, no lo vuelvas a crear.

9. Verificar contraseña y permisos por TCP

Qué vamos a hacer y para qué

Demostrar que la cuenta limitada realmente puede conectarse con su contraseña.

Antes de hacerlo, entiende esto

TCP es la conexión de red que usará la API. Entrar como administrador dentro del contenedor no demuestra que la contraseña del usuario limitado sea correcta. -W hace que psql la solicite sin escribirla en el comando.

Hazlo en este orden

Si tienes psql instalado en tu equipo, desde la Terminal A:

```
psql -h 127.0.0.1 -p 5433 -U medinet_pac_api \
  -d medinet_pac -W -v ON_ERROR_STOP=1 \
  -f scripts/check_runtime_permissions.sql
```

Escribe la contraseña que configuraste para medinet_pac_api. El archivo consulta metadatos de permisos; no muestra pacientes. -h es el destino, -p el puerto y -f el archivo de instrucciones.

En la salida, t significa true (permitido) y f significa false (no permitido). Debe mostrar CONNECT y USAGE permitidos; SELECT, INSERT y UPDATE en patients permitidos; DELETE y TRUNCATE denegados. La secuencia solo debe permitir USAGE; alembic_version debe estar denegada.

Cómo saber si funcionó

current_user es medinet_pac_api y current_database es medinet_pac. El comando termina sin error y los permisos coinciden.

Si no sale como esperabas

Si dice password authentication failed, no avances suponiendo que salud lo resolverá. Revisa que usaste la contraseña del rol correcto; vuelve a \password si necesitas corregirla. Si dice psql: command not found, usa la alternativa de la página siguiente.

9.1. Alternativa si no tienes psql local

Qué vamos a hacer y para qué

Usar el cliente psql incluido en el contenedor para verificar una conexión TCP interna.

Antes de hacerlo, entiende esto

Esta variante comprueba contraseña dentro del contenedor, en el puerto 5432. No demuestra por sí sola que el puerto 5433 de tu equipo funcione; después lo comprobaremos con solicitudes a la API.

Hazlo en este orden

En la Terminal A: copia únicamente el script de metadatos al contenedor:

```
podman exec -i medinet_pac_db sh -c \
  'cat > /tmp/pac_check_permissions.sql' \
  < scripts/check_runtime_permissions.sql
```

No es un archivo de contraseñas. Luego ejecuta el cliente interactivo:

```
podman exec -it medinet_pac_db psql \
  -h 127.0.0.1 -p 5432 -U medinet_pac_api \
  -d medinet_pac -W -v ON_ERROR_STOP=1 \
  -f /tmp/pac_check_permissions.sql
```

Introduce la contraseña del rol limitado. Compara los permisos con la página anterior. No necesitas hacer ambas variantes si ya comprobaste la conexión desde el host.

> Diferencia clave: 5432 es el puerto de PostgreSQL dentro del contenedor; 5433 es el publicado en tu equipo. Cambiar uno por el otro sin considerar desde dónde te conectas causa errores.

Cómo saber si funcionó

La conexión interna autentica a medinet_pac_api y sus permisos son los esperados.

Si no sale como esperabas

Si falla, corrige la contraseña antes de arrancar la API. Si la verificación interna funciona y la API no conecta, revisa publicación del puerto, host y parámetros del lanzador.

10. Arrancar la API y dejarla abierta

Qué vamos a hacer y para qué

Poner en marcha el programa que recibe solicitudes de pacientes.

Antes de hacerlo, entiende esto

El lanzador app.run_local pide la contraseña oculta, configura la conexión del usuario limitado y ejecuta el servidor HTTP. No usa la contraseña administrativa de .env. El servidor ocupa la terminal mientras está activo.

Hazlo en este orden

En la Terminal A, con .venv activo:

```
python -m app.run_local --port 8001
```

Escribe la contraseña de medinet_pac_api y presiona Enter. Deja esta terminal abierta. Para no detener el servidor, el resto de las órdenes del sistema las haremos en una **Terminal B**.

En el navegador abre:

```
http://127.0.0.1:8001/health
http://127.0.0.1:8001/docs
```

La primera dirección comprueba el proceso. La segunda abre Swagger, una página para usar la API sin construir todavía un frontend.

> El 8001 es el puerto HTTP de esta receta. El puerto de PostgreSQL sigue siendo 5433. Si decides usar otro puerto HTTP, debes cambiar todas las direcciones del navegador de forma consistente.

Cómo saber si funcionó

La consola indica que el servidor está iniciado. /health muestra {"status":"ok"} y /docs muestra las operaciones de pacientes.

Si no sale como esperabas

Si 8001 está ocupado, identifica tu instancia previa o inicia con --port 8002 y usa 8002 en todas las URL. Si salud funciona, todavía falta comprobar PostgreSQL: lo haremos al crear o consultar pacientes.

11. Aprender Swagger y validar un RUN

Qué vamos a hacer y para qué

Enviar nuestra primera solicitud sin guardar un paciente todavía.

Antes de hacerlo, entiende esto

Swagger muestra las operaciones de la API. POST envía datos; GET consulta; PATCH cambia algunos campos. JSON es el formato de texto con llaves y pares nombre:valor. Usa comillas dobles y no agregues comentarios dentro del JSON.

Hazlo en este orden

En el navegador, en /docs:

1. Busca POST /patients/validate-run y haz clic para desplegarlo.
2. Pulsa Try it out.
3. Borra el ejemplo del cuadro Request body y pega:

```
{"run":"12.345.678-5"}
```

4. Pulsa Execute. Mira **Server response**, su código y Response body. No confundas los ejemplos de la documentación con la respuesta que acaba de devolver el servidor.

5. Cambia el RUN a 12345678-4 y vuelve a ejecutar.

Cómo saber si funcionó

Primero: código 200 con valid: true y normalized_run: "12345678-5". Segundo: 200 con valid: false y normalized_run: null.

Si no sale como esperabas

Un 422 indica que el cuerpo no tiene la forma o el tipo esperado. Revisa comillas y que run sea texto. Esta operación comprueba formato y módulo 11; no acredita que la persona exista y no demuestra acceso a la base.

12. Crear un paciente inventado

Qué vamos a hacer y para qué

Guardar un registro mediante la API y comprobar por primera vez una escritura real en PostgreSQL.

Antes de hacerlo, entiende esto

La API valida y normaliza los datos antes de guardarlos. El ID lo genera la base: no lo enviamos. Location indica la dirección del recurso creado.

Hazlo en este orden

En Swagger: abre POST /patients, pulsa Try it out y reemplaza el cuerpo por:

```
{
  "run": "12.345.678-5",
  "nombres": "Ana María Sintética",
  "apellido_paterno": "Muñoz",
  "apellido_materno": "O'Neill",
  "fecha_nacimiento": "1990-05-10",
  "prevision": " fonasa ",
  "email": "Ana.PAC@EXAMPLE.COM",
  "telefono": "+56 (9) 1234-5678",
  "direccion": "Calle Sintética 123"
}
```

Pulsa Execute y anota el id de la respuesta. Luego repite exactamente la solicitud para comprobar el rechazo de duplicados.

Cómo saber si funcionó

Primer intento: 201 y un ID. RUN queda 12345678-5; previsión FONASA; teléfono 56912345678. Al repetir: 409 porque el RUN ya existe.

Si no sale como esperabas

Si el primer intento da 409, puede ser una demostración previa: usa GET /patients filtrando por ese RUN para obtener el ID. Si devuelve 503, revisa la conexión PostgreSQL. No borres datos para repetir la receta.

13. Consultar lo que guardamos

Qué vamos a hacer y para qué

Leer el paciente por su ID y aprender a buscarlo en un listado.

Antes de hacerlo, entiende esto

El ID es el número interno de referencia. Un filtro reduce los resultados. La paginación divide el listado en grupos. Los filtros enviados se combinan: el registro debe cumplirlos todos.

Hazlo en este orden

Primera consulta:

1. Abre GET /patients/{patient_id} en Swagger.
2. Pulsa Try it out.
3. En patient_id escribe el ID que copiaste.
4. Pulsa Execute y comprueba los datos.

Segunda consulta: abre GET /patients, pulsa Try it out y completa:

```
apellido_paterno: mUÑ
apellido_materno: o'
prevision: FONASA
page: 1
page_size: 1
```

Estos son valores para los campos de Swagger, no un comando ni un JSON. Deja run sin enviar en esta búsqueda. Pulsa Execute.

items contiene los pacientes de la página; total cuenta todos los que cumplen los filtros; page indica la página solicitada; page_size indica el tamaño solicitado.

Cómo saber si funcionó

Ambas respuestas son 200 y aparece Ana María Sintética. mUÑ coincide con Muñoz porque no distingue mayúsculas; Munoz no coincide porque las tildes sí importan.

Si no sale como esperabas

Si por ID devuelve 404, comprueba que copiaste el ID real. Si el listado queda vacío, quita filtros y prueba de a uno. Una página 999 puede devolver items vacío sin ser un error.

14. Cambiar solo un dato y borrar otro

Qué vamos a hacer y para qué

Actualizar el teléfono y quitar el correo sin reemplazar todo el registro.

Antes de hacerlo, entiende esto

PATCH representa una modificación parcial. Omitir un campo conserva el valor anterior. null es un valor JSON especial que permite vaciar un campo opcional; no es la cadena de texto "null".

Hazlo en este orden

En Swagger:

1. Abre PATCH /patients/{patient_id}.
2. Pulsa Try it out y escribe el ID del paciente.
3. Pega este cuerpo y pulsa Execute:

```
{"telefono":"987654321","email":null}
```

Ahora repite GET /patients/{patient_id}. Así verificas que el cambio quedó guardado, no solo que se mostró en la respuesta del PATCH.

Como prueba de rechazo, intenta PATCH con este cuerpo:

```
{"nombres":null}
```

Y luego con un cuerpo vacío:

```
{}
```

Cómo saber si funcionó

El PATCH válido devuelve 200; el teléfono cambia, email pasa a null y nombres/apellidos se conservan. El GET posterior confirma lo mismo. Los dos PATCH inválidos devuelven 422.

Si no sale como esperabas

Si devuelve 404, revisa el ID. Si devuelve 409 al cambiar un RUN, ya pertenece a otro paciente. No envíes id, created_at o updated_at: los controla el sistema.

15. Entender las validaciones y los errores

Qué significan los códigos

- **200:** solicitud atendida correctamente.
- **201:** paciente creado.
- **404:** el paciente pedido no existe.
- **409:** conflicto, por ejemplo RUN repetido.
- **422:** datos o parámetros inválidos.
- **503:** problema de disponibilidad de la conexión a la base.
- **500:** fallo interno; la respuesta oculta detalles técnicos sensibles.

Pruebas pequeñas que puedes repetir

En POST /patients cambia solo un campo a la vez respecto del ejemplo: un RUN con DV incorrecto, previsión OTRA, una fecha futura o un campo extra como id. Cada caso debe dar 422. No necesitas que la escritura se complete para comprobar una validación.

En GET por ID, 0 debe dar 422. Un ID positivo que no exista debe dar 404; no asumas que un número concreto está libre en todas las bases.

Reglas implementadas

RUN admite cuerpo de 7 u 8 dígitos y verifica módulo 11, incluidos K y 0. Previsión admite FONASA, ISAPRE, CAPREDENA o DIPRECA. Se limpian espacios en nombres y se conservan tildes, ñ, apóstrofos y guiones. Correo valida sintaxis sin consultar DNS; teléfono guarda de 1 a 15 dígitos sin añadir país.

Los filtros por apellido usan prefijo literal; los caracteres % y _ no amplían la consulta. page_size está entre 1 y 100 y su valor predeterminado es 20.

> Una validación correcta del RUN no prueba identidad. Tampoco las pruebas de JSON equivalen a comprobar el renderizado seguro de un futuro frontend. Todavía no hay interfaz de pacientes ni autenticación de usuarios HTTP.

16. Ejecutar las pruebas sin tocar tus datos

Qué vamos a hacer y para qué

Comprobar de forma automática muchos casos en una base exclusiva de pruebas.

Antes de hacerlo, entiende esto

Una prueba automatizada ejecuta acciones y compara resultados. El runner prepara un PostgreSQL temporal con contraseñas aleatorias y usuarios separados. Usa puerto 55434, distinto del 5433 de desarrollo.

Hazlo en este orden

En la Terminal B: entra a tu proyecto y activa su entorno:

```
cd "/ruta/de/tu/MediNet-Core"
source .venv/bin/activate
podman image exists docker.io/library/postgres:18-alpine
```

El último comando puede terminar sin imprimir nada. Si falta la imagen, descárgala:

```
podman pull docker.io/library/postgres:18-alpine
```

Ejecuta después:

```
python scripts/run_pac_tests.py
```

El runner necesita Podman, incluso si administras desarrollo con Docker. Usa un contenedor nuevo llamado medinet_pac_credentials_test, no .env ni el volumen de desarrollo. Conserva ese contenedor al finalizar para inspección.

Cómo saber si funcionó

La salida termina con todas las pruebas aprobadas y sin fallos. Hay un reporte histórico de 119 aprobadas; registra el resultado real de tu copia.

Si no sale como esperabas

Si no tienes Podman operativo, esta suite no se puede ejecutar tal como está: deja registrada esa limitación. No sustituyas la base de pruebas por tu base de desarrollo ni ejecutes pytest contra pacientes reales.

16.1. Si quieres repetir la suite

Qué vamos a hacer y para qué

Liberar únicamente el contenedor temporal de una ejecución anterior y volver a empezar.

Antes de hacerlo, entiende esto

El runner se niega a reutilizar un contenedor existente para no confundir entornos. Eliminar este contenedor destruye solo sus datos temporales; antes hay que identificarlo.

Hazlo en este orden

En la Terminal B: inspecciona el contenedor de pruebas:

```
podman inspect medinet_pac_credentials_test --format \
  '{{.Id}} {{.Config.Labels}} {{json .Mounts}} {{json .NetworkSettings.Ports}}'
```

Comprueba nombre e identidad: debe ser el creado por tu suite, con etiqueta medinet.task=pac-credentials-tests, puerto publicado 55434 y sin el volumen de desarrollo. Si algo no coincide, no lo elimines.

Solo después de reconocerlo, ejecuta:

```
podman rm -f medinet_pac_credentials_test
python scripts/run_pac_tests.py
```

rm elimina ese contenedor y -f fuerza su parada si seguía activo. No se debe cambiar el nombre del comando a medinet_pac_db, que es tu base de desarrollo.

Cómo saber si funcionó

La nueva ejecución crea su propio servidor temporal y finaliza con las pruebas aprobadas.

Si no sale como esperabas

Si la inspección indica un recurso que no reconoces, revisa con el equipo antes de borrarlo. No uses limpiezas globales ni elimines volúmenes para repetir la suite.

17. Qué archivos hacen cada trabajo

Para entender el código que acabas de integrar

- **app/main.py:** construye la aplicación, registra las rutas y centraliza respuestas de error.
- **app/routers/patients.py:** recibe las solicitudes HTTP y define qué operación se ejecuta.
- **app/schemas.py:** define qué datos se aceptan y qué estructura tienen las respuestas.
- **app/validators/run.py:** normaliza el RUN y comprueba su dígito verificador.
- **app/services/patients.py:** consulta, inserta y actualiza; aplica filtros y maneja transacciones.
- **app/models.py:** representa la tabla patients y sus restricciones mediante SQLAlchemy.
- **app/db_base.py:** contiene la base declarativa de los modelos.
- **app/database.py:** conecta con el rol limitado y entrega una sesión por solicitud.
- **app/admin_database.py:** prepara la conexión administrativa usada para migraciones.
- **app/crear_tablas.py:** conserva el comando anterior, ahora respaldado por Alembic.
- **app/run_local.py:** pide la contraseña y arranca el servidor.
- **migrations/:** registra cómo evolucionó el esquema sin recrearlo.
- **scripts/:** incluye SQL de permisos y preparación de las pruebas aisladas.
- **tests/:** comprueba contrato, persistencia, concurrencia, permisos y migraciones.

Dos conceptos útiles

Transacción: conjunto de cambios que se confirman juntos. Si algo falla, rollback deshace lo no confirmado. **UNIQUE:** regla de la base que impide RUN duplicados incluso cuando dos solicitudes llegan al mismo tiempo.

Las consultas usan parámetros para los valores. Los mensajes de error no devuelven SQL, entradas personales ni detalles de conexión. Estas medidas forman parte del trabajo ya implementado.

18. Dejar al equipo en el mismo punto

Lista de llegada: marca cada comprobación

- [ ] Incorporé el mismo Paso3_archivos.zip y conservé mi .env.
- [ ] Mis dependencias se instalaron sin incompatibilidades.
- [ ] PostgreSQL está disponible en el motor correcto.
- [ ] Alembic muestra 0002 y no detecta cambios pendientes.
- [ ] La cuenta medinet_pac_api autentica y sus permisos coinciden.
- [ ] La API abre /health y /docs en mi puerto elegido.
- [ ] Creé o recuperé el paciente sintético y lo consulté por ID.
- [ ] Probé PATCH y confirmé el resultado con un GET posterior.
- [ ] Comprobé 409 por duplicado y 422 por entrada inválida.
- [ ] Ejecuté la suite o registré por qué no pude ejecutarla.

Plantilla para compartir resultados

```
Integrante:
Fecha y sistema operativo:
Versión de Python:
Motor de contenedores:
Puerto de BD / puerto de API:
Alembic current / check:
Conexión con rol limitado:
Pruebas Swagger realizadas:
Resultado de la suite:
Problema pendiente, si existe:
```

Comparte esta información o capturas con datos inventados. No compartas .env, contraseñas, respaldos ni información personal. Tras acordar la integración, el equipo podrá registrar y compartir una revisión Git común.

Cómo apagar sin borrar

En la Terminal A pulsa Ctrl+C para detener la API. En la Terminal B ejecuta:

```
podman-compose stop
```

stop conserva los datos. Para el próximo inicio: enciende la base, activa .venv y repite el lanzador. No necesitas reaplicar el script de roles ni las migraciones cada vez.

19. Si te atascas, revisa esto primero

“No existe el archivo o directorio”

Ejecuta pwd y ls. Confirma que estás en MediNet-Core y que copiaste el contenido correcto del ZIP.

“No module named…” o comando no encontrado

Activa .venv en esa terminal y revisa la instalación de requirements-dev.txt. Si falta psql local, usa la alternativa del paso 9.1.

“password authentication failed”

Identifica qué cuenta estás usando. Alembic usa medinet_pac y .env; la API usa medinet_pac_api y la contraseña del prompt. Cambiar .env no cambia la contraseña de PostgreSQL.

Salud funciona, pero pacientes falla

Salud comprueba el proceso HTTP, no la base. Revisa el paso 9, que PostgreSQL esté encendido y que el lanzador use host 127.0.0.1, puerto 5433 y base medinet_pac.

La dirección no abre o el puerto está ocupado

Mira la Terminal A: el proceso debe seguir abierto. Comprueba el puerto usado. 8001 es API, 5433 es PostgreSQL del host y 55434 es pruebas. No detengas servicios ajenos sin identificarlos.

“Role already exists”

El rol ya estaba creado. No repitas el SQL: vuelve a comprobar contraseña y permisos. En el equipo de origen ya se había aplicado.

La migración dice que la tabla no coincide

Detente ahí: tu esquema puede diferir del paso 2 esperado. Conserva el error, sin secretos, y compáralo con el equipo. No borres patients ni el volumen para hacer que el comando pase.

> Cuando pidas ayuda, comparte el paso exacto, el comando sin secretos y el mensaje de error. Indica también si usas Docker o Podman y qué puerto elegiste.

20. Qué está hecho y qué sigue pendiente

Lo que esta entrega integra

Cinco operaciones PAC: crear, listar, consultar por ID, actualizar parcialmente y validar RUN. Validaciones y normalización, filtros y paginación, transacciones y manejo de errores, migraciones 0001/0002, índices de apellidos, separación de credenciales y pruebas aisladas.

Qué evidencia teníamos antes de esta receta

El informe inicial registra 87 pruebas aprobadas; una ejecución posterior reportó 119. Las migraciones se aplicaron a desarrollo conservando filas y restricciones, y Alembic no detectó cambios pendientes. El rol limitado se aplicó y sus permisos se verificaron por conexión local dentro del contenedor.

La candidata en 8001 respondió a salud y OpenAPI. Una prueba TCP independiente rechazó la contraseña proporcionada; quedó pendiente verificar la credencial correcta y la conexión realmente utilizada por esa instancia. Por eso esta receta incluye pasos explícitos de autenticación y operaciones sobre pacientes.

> Al redactar esta edición no se repitió la suite ni se cambiaron datos o permisos de desarrollo. Los resultados esperados son lo que cada integrante debe verificar en su propio equipo, no un registro de una ejecución nueva.

Lo que todavía no implementamos

Frontend de pacientes, comprobación de renderizado en navegador, autenticación y autorización HTTP, contenedor de API, pipeline, análisis SAST/SCA, HTTPS, AWS y contrato de integración con los demás módulos. Campos y largos definitivos deben acordarse con el docente.

La cuenta de PostgreSQL limita al programa frente a la base; no determina qué persona puede entrar a la API. Esta entrega se mantiene como demostración local con datos sintéticos.

Dónde seguir leyendo

README.md contiene el contrato y el estado actualizado. docs/paso3-validacion.md conserva el informe de la primera entrega; algunos pendientes de ese informe fueron tratados después y se explican en README. El repositorio de origen aún no tiene commits ni push.